Development of an Automatic Design Optimization Platform for a 2D Heat-Conduction System Using FreeFEM++

HEAT-COND benchmark with FreeFEM++ and Python

Laurent Zhu Alexandre Le Droucpeet

Optimization and Numerical Analysis (MATH6304P-260-M01), Spring 2026
Shanghai Jiao Tong University, SPEIT

Abstract

We develop a compact, automatic design-optimization platform for the 2D steady-state HEAT-COND heat-conduction benchmark, coupling a FreeFEM++ P_1 finite-element solver with a Python optimization driver. The study is organized in two parts. **Part I** solves the original benchmark: maximizing the mean fin temperature J over the conductivities and the Biot number. Through systematic studies we show that this objective is smooth and unimodal with a *boundary* optimum at $\mathbf{x}^\star = (1, 1, 1, 1, 1, 0.01)$, giving $J^\star \approx 0.730$ at mesh density 50 (an $\approx 850\%$ improvement over the uniform design), and that **Nelder–Mead** is among the most efficient optimizers (Bayesian optimization reaches the same optimum in still fewer evaluations). **Part II** enriches the problem into a realistic engineering design: each fin now carries a *discrete material* (conductivity, price, density) and a *continuous geometry* (thickness, length), and we maximize a multi-criteria objective $F = Q - \lambda_{\text{cost}}\,\text{Cost} - \lambda_{\text{mass}}\,\text{Mass}$. On this 16-dimensional, mixed integer/continuous problem we apply the same methods as in Part I together with **Bayesian optimization**; they land within a few percent, and Bayesian optimization attains the best objective using by far the fewest evaluations, so we retain it as the default. We characterize its convergence both per evaluation and in CPU time, and a performance–cost Pareto sweep shows that the material cost can be cut by about 90% for a moderate performance loss.

1 Introduction and Project Objective

This project develops a compact, automatic design-optimization platform for a two-dimensional steady-state heat-conduction problem, the HEAT-COND benchmark drawn from the reduced-basis literature [1]. The platform integrates, within a single reproducible workflow, the five required components: a finite-element solver written in FreeFEM++, an objective-evaluation module, a Python optimization driver, a graphical user interface, and a numerical-analysis framework for comparing optimization strategies.

The report follows a deliberate two-step structure. **Part I** answers the original benchmark question with the simple objective J (the mean fin temperature) and identifies the best-suited optimizer *and why*. **Part II** then enriches the design space with the choice of material (with its price and density) and the dimensioning of each fin (thickness and length), turning the academic benchmark into a genuine multi-objective engineering design and re-examining which optimization strategy is best on this harder, mixed and non-smooth landscape. A single structural idea ties the two parts together: the geometry of the objective (smooth and unimodal versus mixed and non-smooth) dictates which optimizer wins.

2 Mathematical Model (Initial Problem)

2.1 Geometry

The computational domain $\Omega \subset [0,1]^2$ is a comb-shaped region consisting of a central vertical *spine* of width $w_s = 0.10$ and five pairs of horizontal *fins* of thickness $t_f = 0.06$, centred at $y \in \{0.16, 0.32, 0.48, 0.64, 0.80\}$. The inner spine edges are at $x_g = (1 - w_s)/2 = 0.45$ and $x_d = x_g + w_s = 0.55$. Two boundary regions are distinguished: Γ_{Base}, the bottom edge of the spine $\{(x, 0) : x \in [x_g, x_d]\}$, and Γ_{Fin}, the remainder of $\partial\Omega$ (the spine top, the outer fin tips, and all fin flanks). The domain, its boundary labels and the P_1 mesh are shown in Fig. 1.

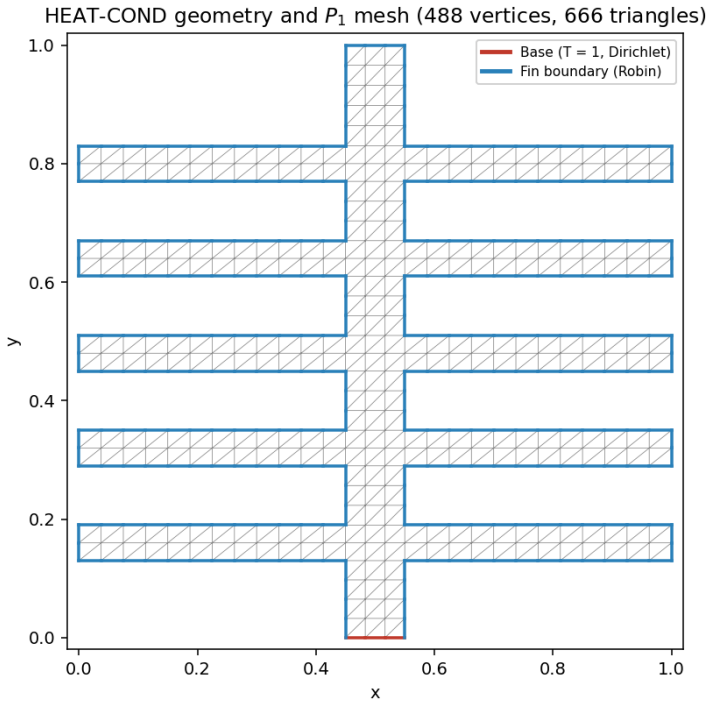


Figure 1: HEAT-COND geometry and P_1 mesh (density 25). The red base Γ_{Base} carries the Dirichlet condition $T = 1$; the blue boundary Γ_{Fin} carries the Robin condition. The central spine has conductivity $k = 1$; each fin i has conductivity k_i.

2.2 Governing Equation

The dimensionless temperature T satisfies the steady-state heat-conduction equation with mixed (Dirichlet–Robin) boundary conditions:

$$\begin{cases} -\nabla \cdot \big(k(\mathbf{x})\, \nabla T\big) = 0 & \text{in } \Omega, \\ T = 1 & \text{on } \Gamma_{\text{Base}}, \\ k(\mathbf{x})\, \dfrac{\partial T}{\partial n} + \text{Bi}\, T = 0 & \text{on } \Gamma_{\text{Fin}}. \end{cases} \tag{1}$$

The conductivity $k(\mathbf{x})$ is piecewise constant: $k = 1$ in the spine, and $k = k_i$ $(i = 1, \ldots, 5)$ in fin i, counted bottom to top. The Biot number Bi controls the convective coupling on Γ_{Fin}.

2.3 Objective and Optimization Problem

The objective is to maximize the average temperature on the fin boundary,

$$J(\mathbf{x}) = \frac{1}{|\Gamma_{\text{Fin}}|} \int_{\Gamma_{\text{Fin}}} T \, \mathrm{d}\Gamma, \tag{2}$$

over $\mathbf{x} = (k_1, \ldots, k_5, \mathrm{Bi}) \in \mathbb{R}^6$ with $k_i \in [0.1, 1.0]$ and $\mathrm{Bi} \in [0.01, 1.0]$: $\max_{\mathbf{x}} J(\mathbf{x})$ subject to box constraints. Physically, a high J means the spine transfers the heat injected at the base ($T = 1$) efficiently to the peripheral fin surfaces.

3 Numerical Formulation and FreeFEM++ Implementation

3.1 Mesh Generation

The mesh is constructed explicitly with FreeFEM's `buildmesh` rather than by truncating a background grid, so that element edges align exactly with the geometry. The boundary is described as a closed, counter-clockwise polyline of 44 segments / 45 points, traversed by a single indexed `border`; the number of subdivisions of each segment is proportional to its Euclidean length times a global density `meshsize`. Two boundary labels are assigned: **label 1** for the base segment (Dirichlet) and **label 2** for the rest of the boundary (Robin). At density 50 the mesh contains 1151 vertices and 1760 triangles (419 vertices at density 25).

3.2 PDE Solver

`solver.edp` reads its parameters through `getARGV`. The weak form is assembled in the P_1 space V_h: find $T \in V_h$ with $T = 1$ on Γ_{Base} such that, for all test functions v,

$$a(T, v) = \int_{\Omega} k \, \nabla T \cdot \nabla v \, \mathrm{d}\Omega + \int_{\Gamma_{\mathrm{Fin}}} \mathrm{Bi} \, T \, v \, \mathrm{d}\Gamma = 0. \tag{3}$$

The Dirichlet condition is imposed strongly via `on(1, T = 1)`; the Robin condition enters naturally as the boundary integral over label 2. The objective $J = \int_{\Gamma_2} T \, \mathrm{d}\Gamma / \int_{\Gamma_2} 1 \, \mathrm{d}\Gamma$ is computed inside the solver and printed to stdout as `Objective_J=<value>`, removing intermediate text files.

3.3 Python Coupling and Verification

The Python layer drives FreeFEM through `subprocess`: it caches meshes and parses the objective from stdout. To verify the implementation, the P_1 weak form was re-implemented in pure NumPy and compared against FreeFEM on identical cached meshes: at density 50 the two solvers agree to six significant digits at the optimal corner ($J(1, 1, 1, 1, 1, 0.01) = 0.730396$ in both), and the vertex/triangle counts match exactly. This cross-check validates the operator and the objective.

Part I. Initial problem: maximizing the mean fin temperature J

4 Optimization Strategy (Part I)

The driver wraps every algorithm behind a uniform interface returning a normalized dictionary (`best_J`, `best_x`, `n_eval`, `time`, full `history`); random methods use a fixed seed. We compare **four methods**, spanning the main families:

- **Nelder–Mead (NM)**: local, deterministic simplex in dimension 6, gradient-free, sensitive to the start, very cheap.
- **Differential Evolution (DE)**: global, population-based, gradient-free, robust but the most expensive.
- **Adam → L-BFGS-B**: a two-phase gradient hybrid, Adam (finite-difference gradient, per-coordinate adaptive step) drives the iterate into a good region, then L-BFGS-B refines with native bound handling.

- **Bayesian optimization (BO)**: a surrogate-based global method (Gaussian-process model with Expected Improvement) aimed at reaching the optimum in as few expensive evaluations as possible; it handles the discrete materials of Part II natively as integer dimensions.

Because the objective is a black-box PDE solve, the gradient phase uses central finite differences, costing $2 \times 6 = 12$ extra evaluations per gradient in 6D. The experimental design is implemented as several self-contained studies.

5 Numerical Results for the Initial Problem

The reference initial design is $\mathbf{x}_0 = (0.5, \ldots, 0.5)$, for which $J_0 \approx 0.077$. Unless stated otherwise, comparisons use mesh density 25; the ranking is identical at density 50.

5.1 Comparison of the Methods and the Optimal Design

Table 1: Comparison of the methods from the common start $\mathbf{x}_0 = 0.5$ (density 25).

Method	$J^\star$	n_{eval}	Time (s)	Design $\mathbf{x}^\star$
Nelder–Mead	**0.73410**	**90**	0.16	$(1, 1, 1, 1, 1, 0.01)$
Adam → L-BFGS	0.73410	367	0.66	$(1, 1, 1, 1, 1, 0.01)$
Differential Evolution	0.72068	468	0.81	$(0.80, 0.66, 0.58, 0.62, 0.46, 0.01)$
Bayesian opt.	0.73410	60	0.2	$(1, 1, 1, 1, 1, 0.01)$

Three of the four methods reach the global optimum $\mathbf{x}^\star = (1, 1, 1, 1, 1, 0.01)$, a vertex of the admissible box (all conductivities at their upper bound, the Biot number at its lower bound), giving $J^\star \approx 0.730$ at density 50, an $\approx 850\%$ improvement over J_0; only Differential Evolution stops short at $J \approx 0.721$ with a non-uniform design, its population budget being too small here. When several methods reach the *same* optimum, what distinguishes them is the *cost* of getting there. We measure that cost by the number of objective evaluations, because each evaluation is one PDE solve, the expensive operation; for Nelder–Mead, the gradient hybrid and Differential Evolution this count is essentially proportional to wall-clock time. By that measure Bayesian optimization is the most efficient, reaching the corner in only 60 evaluations against 90 for Nelder–Mead and 367 for the hybrid (Fig. 2). The one caveat is that Bayesian optimization also fits a surrogate at each step; this overhead is negligible against a true FreeFEM solve but not against the very fast NumPy reference solver, so its real advantage lies in the expensive evaluations it saves. The effect on the temperature field is shown in Fig. 3: the initially cold fins (T down to 0.003 at the tips) become hot ($T \geq 0.601$).

5.2 Mesh-Sensitivity

$J^\star$ decreases monotonically and stabilizes near 0.73 as the mesh is refined, indicating approximate mesh-independence by density 50–60; coarse meshes slightly over-estimate the objective. The wall-clock time grows steeply with density (Fig. 4).

Table 2: Optimized objective (Nelder–Mead) vs mesh density.

Mesh density	15	25	40	60
$J^\star$	0.7407	0.7341	0.7311	0.7295

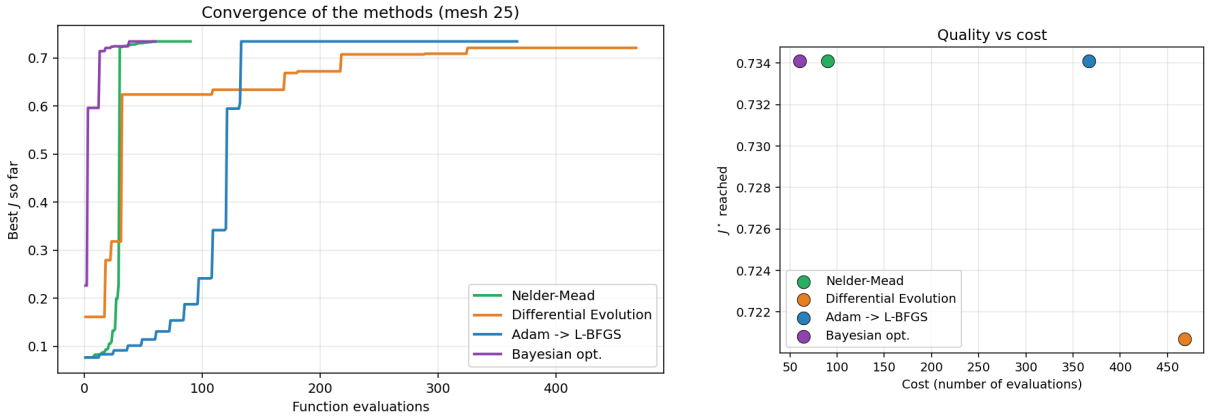


Figure 2: Convergence (left) and quality-vs-cost trade-off (right). Nelder–Mead and the gradient hybrid reach the optimal corner (Nelder–Mead in the fewest evaluations); Differential Evolution falls short at this budget, and Bayesian optimization is the most sample-efficient.

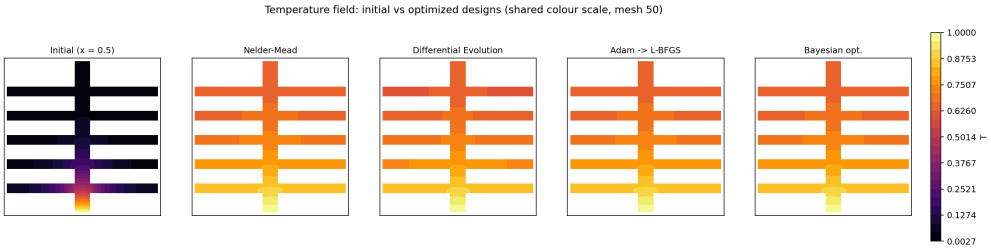


Figure 3: Temperature fields (shared colour scale, mesh 50) for the initial design and the four optimized designs. Optimization raises the minimum fin temperature from 0.003 to about 0.60.

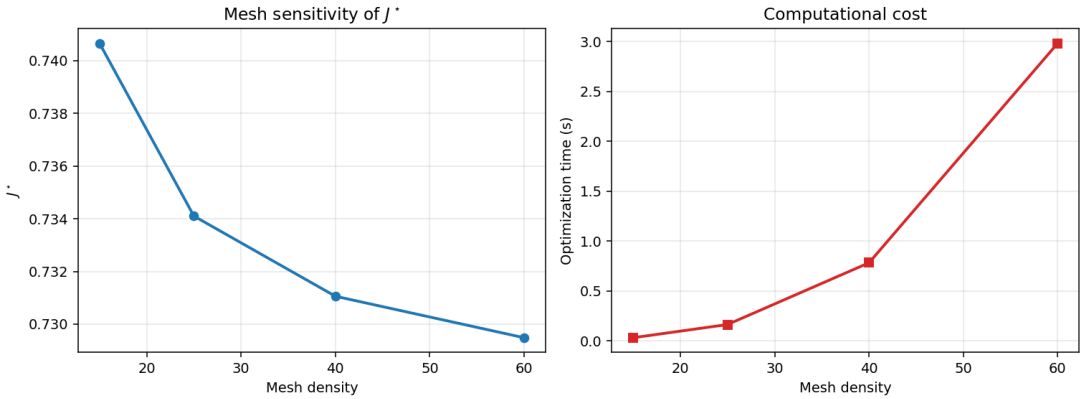


Figure 4: Mesh-sensitivity of $J^\star$ (left) and computational cost (right).

5.3 Robustness

We vary the starting point for the local and gradient methods (NM, Adam) and the random seed for the stochastic methods (DE, Bayesian optimization). Nelder–Mead and the Adam→L-BFGS hybrid reach the corner from every start; Differential Evolution and Bayesian optimization, being stochastic, show a small seed-to-seed spread (a few $\times 10^{-3}$). The objective is effectively unimodal, so the methods are highly reproducible (Fig. 5).

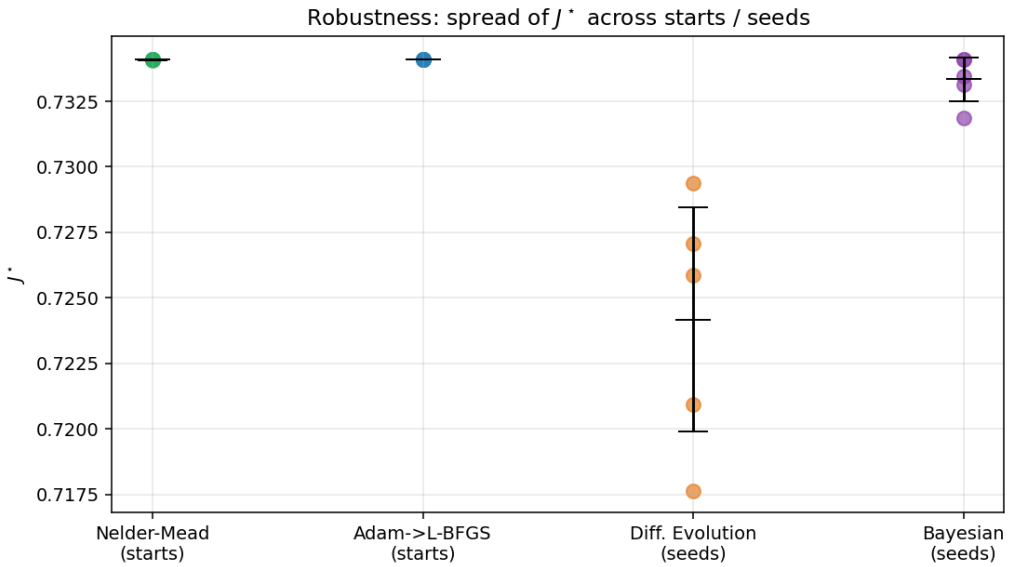


Figure 5: Variability of $J^\star$ across starting points (NM, Adam) and across seeds (DE).

5.4 Gradient Methods: the Hybrid and its Components

This study isolates the contribution of each phase by adding two diagnostic runs from $\mathbf{x}_0 = 0.5$:

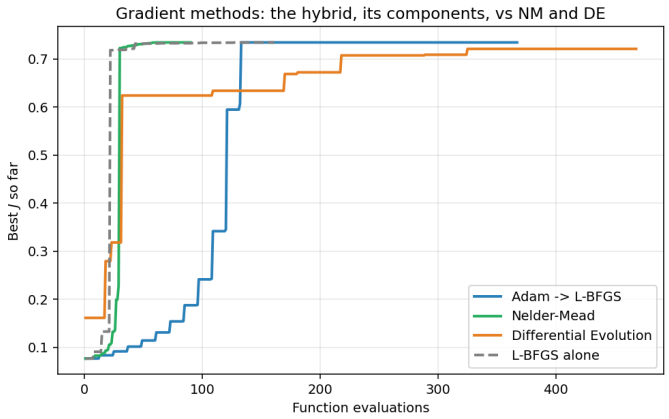
Table 3: Diagnostic runs of the gradient hybrid (density 25).

Variant	$J^\star$	n_{eval}	Reaches the corner
Adam → L-BFGS	0.734100	367	yes
Adam alone	0.734100	360	yes
L-BFGS-B alone	0.734100	161	yes

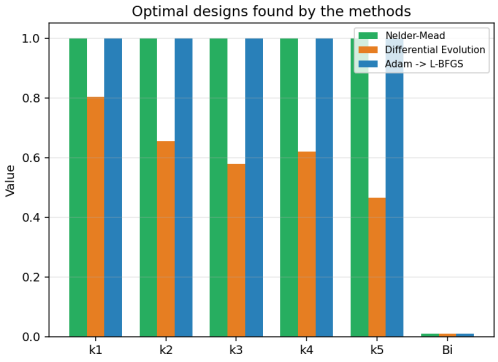
All three gradient variants reach the corner. On this smooth reference solver, scipy's bound-constrained L-BFGS-B reaches it on its own (161 evaluations), so the Adam warm-start is not strictly required here; its benefit shows on noisier or more strongly ill-conditioned objectives, where a pure quasi-Newton step can stall. The objective is markedly ill-scaled: the gradient in the Biot direction dominates those in the conductivity directions, yet the box constraints and the smoothness let the quasi-Newton iterates slide to the corner. Adam contributes the complementary idea of per-coordinate adaptive steps, which normalize this imbalance and make the hybrid robust. Nelder–Mead and the gradient methods converge to the same physical design; Differential Evolution, short of budget, does not (Fig. 6b).

5.5 Conclusion of Part I

The initial HEAT-COND objective is *smooth and unimodal with a boundary optimum* at $\mathbf{x}^\star = (1, 1, 1, 1, 1, 0.01)$, $J^\star \approx 0.730$. On this landscape **Nelder–Mead is the most efficient opti-**



(a)



(b)

Figure 6: (a) Convergence of the hybrid against NM and DE, and of L-BFGS-B alone (dashed), which also reaches the optimum. (b) Optimal designs: Nelder–Mead and the gradient methods converge to $k_i \to 1$, Bi $\to 0.01$; Differential Evolution, short of budget, lands on a non-uniform design.

mizer and the recommended default; Differential Evolution, given a sufficient budget, serves as a global-optimality check, and the gradient methods (Adam→L-BFGS) reach the optimum efficiently. This corner optimum is, however, an artefact of higher conductivity being free, which motivates Part II.

Part II. Enriched problem: material and geometry design (complex objective)

6 Enriched Model: Material, Dimensioning and a Multi-Objective Goal

The Part I optimum sits at a vertex of the box because raising any conductivity carries no cost. A realistic design must pay for what it builds. We therefore enrich the problem on two fronts at once: each fin is assigned a *discrete material* (fixing its conductivity, price and density) and a *continuous geometry* (thickness and length, which physically reshape the mesh).

6.1 Design Vector and Material Catalogue

The design becomes 16-dimensional, grouped by type,

$$\mathbf{x} = [\, m_1 \dots m_5 \;\mid\; t_1 \dots t_5 \;\mid\; \ell_1 \dots \ell_5 \;\mid\; \mathrm{Bi}\,], \tag{4}$$

where $m_i \in \{0, 1, 2, 3\}$ selects fin i's material from the catalogue of Table 4, $t_i \in [0.02, 0.14]$ is its thickness, $\ell_i \in [0.10, 0.45]$ its length, and $\mathrm{Bi} \in [0.01, 1]$. The five m_i are *integer* variables; the eleven remaining variables are continuous.

6.2 Objective: Heat Dissipated versus Cost and Mass

We maximize the scalarized objective

$$F(\mathbf{x}) = Q(\mathbf{x}) - \lambda_{\mathrm{cost}} \sum_{i=1}^{5} \mathrm{price}(m_i)\, A_i - \lambda_{\mathrm{mass}} \sum_{i=1}^{5} \mathrm{density}(m_i)\, A_i, \qquad A_i = 2\, t_i\, \ell_i, \tag{5}$$

Table 4: Material catalogue (illustrative, normalized values). The index m_i fixes a fin's conductivity k, unit price and density.

Index	Material	k	Price/vol.	Density
0	Copper	1.00	9.0	9.0
1	Aluminium	0.60	3.0	2.7
2	Steel	0.25	1.5	7.8
3	Polymer	0.05	0.3	1.2

where A_i is the area of fin pair i (left+right symmetric) and the performance is the *heat dissipated through the fins*,

$$Q(\mathbf{x}) = \int_{\Gamma_{\mathrm{Fin}}} \mathrm{Bi}\, T \, \mathrm{d}\Gamma. \tag{6}$$

We deliberately use Q rather than the Part I mean temperature J. Indeed J *increases* as fins shrink (less lossy surface), which would align performance with low cost and *collapse* the trade-off into a degenerate front. The dissipated heat Q instead grows with surface, conductivity and Biot number, so it genuinely *opposes* cost and mass. Throughout Part II we use $\lambda_{\mathrm{cost}} = 0.05$ and $\lambda_{\mathrm{mass}} = 0.02$ unless a sweep is stated.

6.3 Numerical Treatment and Multi-Fidelity

Because the geometry (t_i, ℓ_i) is now part of the design, the mesh must be *regenerated at every evaluation* (it can no longer be cached once and reused, as in Part I). To keep the cost manageable we adopt a **multi-fidelity** strategy: a global search on a coarse mesh (density ∼18–20), then a local refinement (Nelder–Mead, materials frozen) on a fine mesh (density ∼45–50). Each evaluation's wall-clock time is logged in the run history. To keep the optimizer comparison fast and fully reproducible, it is run on a validated pure-NumPy reference solver (an extension to variable geometry of the cross-check solver of Sec. 3.3): at the default geometry it reproduces the Part I corner value $J = 0.730$ to three digits, so it ranks the methods faithfully while costing only a few milliseconds per evaluation on the coarse mesh.

7 Numerical Results for the Enriched Problem

7.1 Comparison of the Optimization Methods

To keep the methodology consistent across the report, we apply to the enriched problem the *same three optimizers as in Part I* (Nelder–Mead (NM), Differential Evolution (DE) and the Adam→L-BFGS hybrid), together with **Bayesian optimization**. The five discrete material variables are handled by *continuous relaxation with rounding* (the material index is rounded inside the objective), exactly as DE already treated them in Part I, so all four methods operate on the same 16-dimensional continuous box. Table 5 and Fig. 7 report the comparison on the coarse mesh (each method run to convergence).

The four methods land within a few percent of each other ($F^\star = 0.548$–0.569). **Bayesian optimization attains the best objective** ($F^\star = 0.5687$), and does so with **by far the fewest evaluations** (80, against 577–1045 for the others): its Gaussian-process surrogate spends evaluations where they most improve the objective, and it treats the five materials natively as integer dimensions. It reaches a high-performance design (the highest dissipated heat, $Q = 0.62$) by placing copper on the hottest bottom fin and accepting a higher material cost; the net objective still wins. The Adam→L-BFGS hybrid is a close second (0.5667) at a cheaper, lighter design; Nelder–Mead follows (0.559); Differential Evolution trails (0.548), again needing a larger budget

Table 5: The four optimizers on the enriched problem (coarse mesh, $\lambda_{\text{cost}} = 0.05$, $\lambda_{\text{mass}} = 0.02$). F is the maximized objective; Q, cost and mass describe the best design. Times are for the NumPy reference solver.

Method	$F^\star$	Q	Cost	Mass	n_{eval}	Time (s)
Bayesian opt.	**0.5687**	0.6225	0.760	0.791	**80**	0.1
Adam → L-BFGS	0.5667	0.5918	0.369	0.333	1045	1.2
Nelder–Mead	0.5586	0.5899	0.436	0.474	577	1.0
Differential Evolution	0.5484	0.5983	0.612	0.966	720	1.2

while still serving as a *global* check. **We therefore retain Bayesian optimization** as the default method for the detailed studies below: it gives both the best objective and, decisively when each evaluation is an expensive PDE solve, the fewest evaluations.

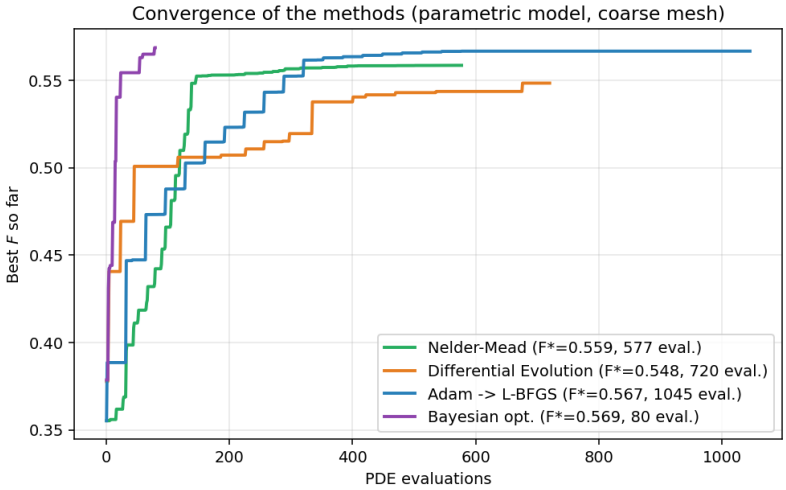


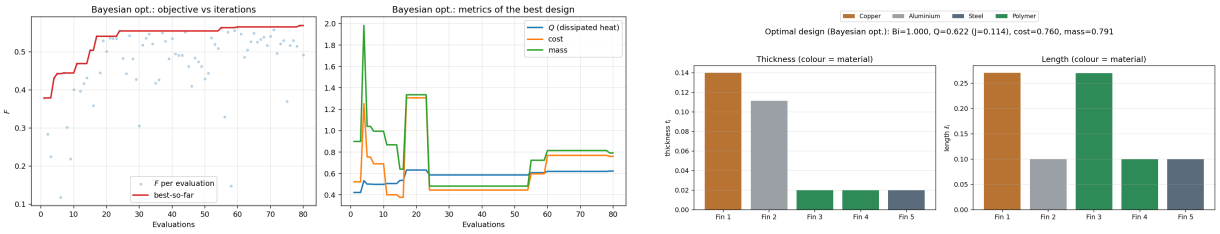
Figure 7: Best-so-far objective F versus PDE evaluations for the four methods (coarse mesh). Bayesian optimization reaches the highest objective in by far the fewest evaluations; the Adam→L-BFGS hybrid is a close second.

7.2 The Optimal Enriched Design

Figure 8a traces the retained run: the objective and its best-so-far envelope, and the joint evolution of the best design's performance Q, cost and mass. The optimal design (Fig. 8b) places *copper*, the most conductive material, on the large bottom fin ($t = 0.14$, $\ell = 0.27$), where the temperature and hence the marginal heat transfer are highest; the upper fins use cheaper aluminium, steel and polymer, and the Biot number saturates at its upper bound ($\text{Bi} = 1.0$, which maximizes the dissipated heat $Q = \int \text{Bi}\, T$). Spending the costly material exactly where it pays off yields the highest performance of all methods ($Q = 0.62$) at a moderate cost (0.76), and the best objective of the comparison ($F = 0.569$).

7.3 Evolution of the Objective for the Retained Method

Having selected Bayesian optimization, we characterize how its objective improves, both *per evaluation* and *in CPU time* (Fig. 9). The run reaches $F^\star = 0.569$ in only 80 evaluations: most of the gain comes in the first dozen surrogate-guided steps, after which the curve flattens, a clear diminishing-returns plateau and a natural stopping point. This is the sample efficiency that mo-



(a) (b)

Figure 8: Retained method (Bayesian optimization). (a) Objective per evaluation with best-so-far envelope, and the performance/cost/mass of the running-best design. (b) Composition of the optimal design: thickness and length per fin, coloured by material (copper on the bottom fin).

tivates a surrogate method on an expensive objective; the Gaussian-process fitting adds a little per-iteration overhead, negligible next to a FreeFEM PDE solve.

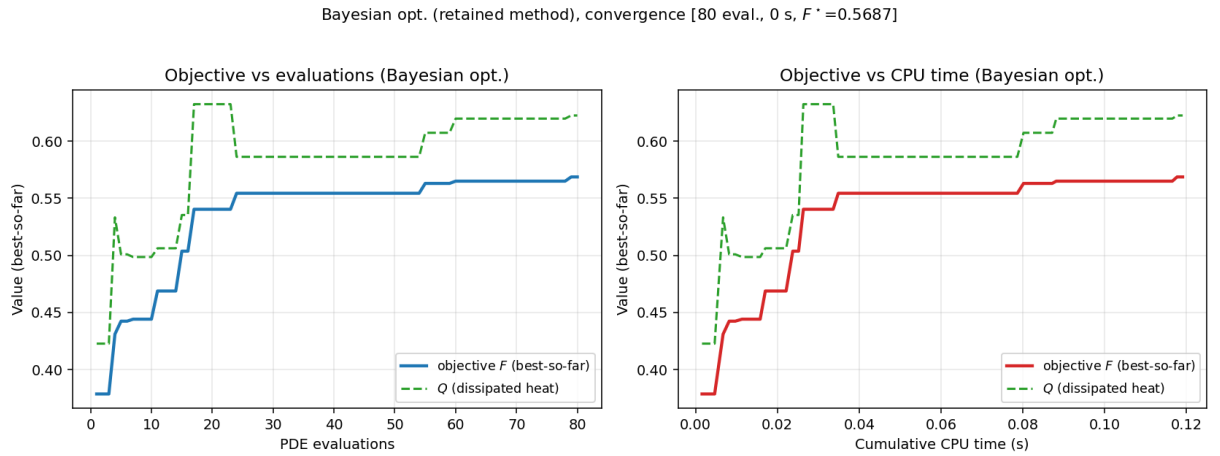


Figure 9: Convergence of the retained method (Bayesian optimization): best-so-far objective F (and the performance Q) versus the number of PDE evaluations (left) and versus cumulative CPU time (right).

7.4 Performance–Cost Pareto Front

To expose the full trade-off we sweep the cost weight λ_{cost} and re-optimize with the retained method for each value, tracing the performance–cost Pareto front (Table 6, Fig. 10). As λ_{cost} grows, the optimizer trades performance for economy in a strongly concave way: from $\lambda = 0$ to $\lambda = 0.20$ the material cost drops by about 91% ($1.21 \to 0.11$) for a $\approx 15\%$ loss of performance Q ($0.63 \to 0.54$). The knee near $\lambda \approx 0.05$–0.10 removes most of the cost for a modest performance loss. This turns the otherwise trivial Part I corner problem into a genuine multi-objective design study with an exploitable front.

7.5 Multi-Fidelity Cost Control

Because the mesh is rebuilt at every evaluation, a fine mesh is accurate but costly, so the idea is to spend the bulk of the search budget on a cheap coarse mesh and only a few evaluations on the accurate fine mesh. The pipeline is a *global-then-local* split across mesh fidelities: a global method on the coarse mesh, then a local method on the fine mesh with the materials frozen so only the continuous geometry is polished.

Table 6: Pareto sweep (retained method, coarse mesh): performance Q, material cost and mass of the optimal design versus the cost weight λ_{cost} (with $\lambda_{\text{mass}} = 0.02$).

λ_{cost}	Q	Cost	Mass	F
0.00	0.6323	1.213	1.235	0.6076
0.02	0.6321	1.174	1.184	0.5850
0.05	0.6225	0.760	0.791	0.5687
0.10	0.6027	0.572	0.614	0.5333
0.20	0.5353	0.107	0.157	0.5108

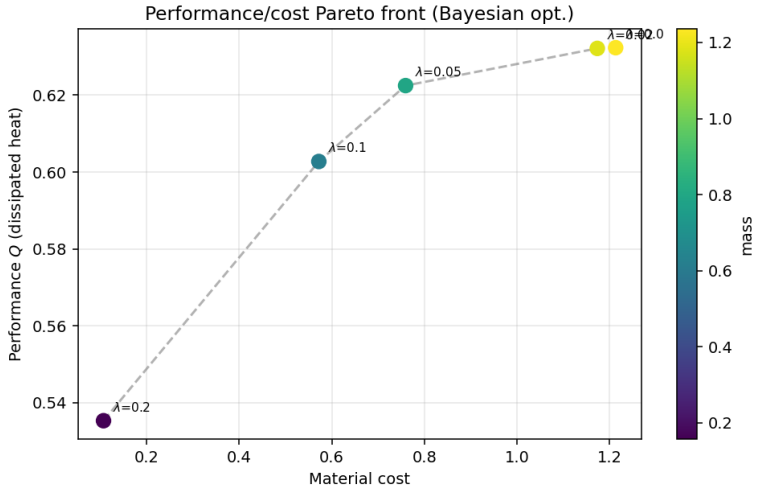


Figure 10: Performance–cost Pareto front of the enriched problem (colour = mass). Each point is an optimum for a given cost weight λ_{cost}; the front is strongly concave near the high-performance end.

The choice of optimizer at each stage is dictated by the stage, not by which method won the single-fidelity comparison. One might expect to reuse Bayesian optimization (the retained method of Section 7.1) and L-BFGS-B here, but neither fits its stage. Bayesian optimization is valuable precisely when each evaluation is *expensive*: it spends real compute fitting a Gaussian-process surrogate to save evaluations. On the coarse mesh an evaluation costs only a few milliseconds, so the surrogate overhead is no longer amortized, and Bayesian optimization would be slower in wall-clock time than a plain population search for the same number of solves. We therefore use Differential Evolution for the global stage: it is cheap per step, explores the discrete materials broadly, and needs no starting point. For the fine stage we use Nelder–Mead rather than L-BFGS-B (or the Adam→L-BFGS hybrid) because remeshing makes the fine-mesh objective slightly non-smooth, which corrupts the finite-difference gradient those methods rely on; a gradient-free local search is more robust to that noise. In short, Bayesian + L-BFGS is the right pairing for a single expensive fidelity, whereas DE + Nelder–Mead is the right pairing once the two-fidelity split has made the global stage cheap and the local stage noisy.

Figure 11 shows the pipeline: a coarse-mesh global search (Differential Evolution, 448 evaluations) followed by a fine-mesh local refinement (Nelder–Mead, density 45, 173 evaluations). The refinement confirms the design; its slightly lower F reflects the fine mesh evaluating Q more accurately (the coarse mesh over-estimates it, exactly as in the Part I mesh study). Without this two-stage strategy, a direct search on the fine mesh would multiply the per-evaluation cost several-fold for no change in the located optimum.

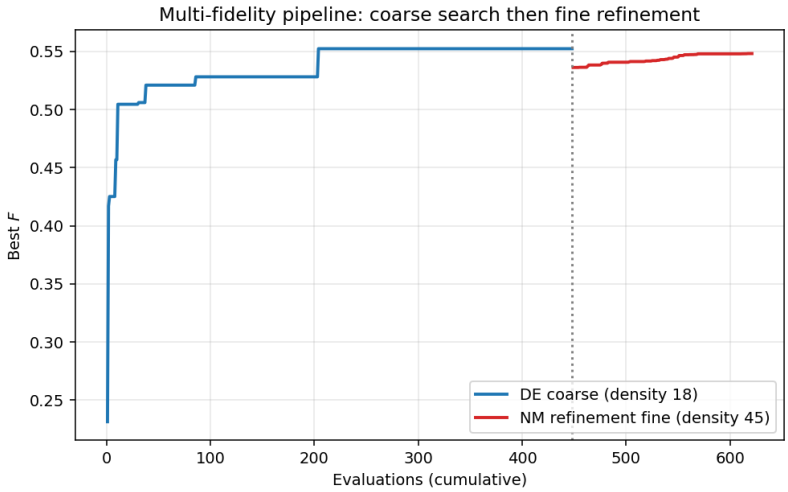


Figure 11: Multi-fidelity pipeline: coarse-mesh global search (DE) followed by fine-mesh local refinement (Nelder–Mead); cumulative evaluations on the horizontal axis, the dotted line marks the hand-off.

8 Analysis and Discussion

Two problems, two regimes. The methodological lesson is that the *same* set of optimizers behave very differently depending on the landscape. In Part I the objective is smooth and unimodal with a boundary optimum, and Nelder–Mead slides straight to the corner, the clear winner: cheapest, most accurate, most reproducible. In Part II the objective is 16-dimensional and mixed (discrete materials) yet *dominated by its continuous geometry*: the four methods perform within a few percent, and *Bayesian optimization* attains the best objective using the fewest evaluations, the Adam→L-BFGS hybrid a close second, then Nelder–Mead and Differential Evolution. The surrogate pays off precisely because each evaluation is expensive; the gradient methods stay strong, and Differential Evolution keeps its role as a global-optimality check. Same platform, same methods: the ranking tightens and shifts as the landscape changes.

Physical reading. Part I says "make everything as conductive as possible and lose as little heat as possible" (a corner). Part II, by pricing material and rewarding dissipated heat, says something an engineer would recognize: concentrate good material and surface where the temperature is highest (the lower fins), economize elsewhere, and let the Biot number rise to shed heat. The optimal design is non-uniform and graded, and the Pareto front quantifies exactly how much performance each saved unit of material costs.

Cost and bottlenecks. The dominant cost is the FreeFEM PDE solve invoked through a subprocess. In Part II the mesh regeneration per evaluation adds to this, which is why the multi-fidelity split and a bounded evaluation budget matter. Two further levers reduce it: a surrogate-based search, examined next, and, for the smooth Part I sub-problem, an analytical/adjoint gradient.

Sample efficiency and Bayesian optimization. Because every evaluation is an expensive PDE solve, the currency that matters most is the number of evaluations spent, not wall-clock time alone. Bayesian optimization targets this directly: it fits a Gaussian-process surrogate to the points already evaluated and chooses each next point by Expected Improvement, and it handles the discrete materials natively through integer dimensions. In both parts it is the most sample-efficient method, making most of its progress in its first evaluations (Figs. 2 and 7). Here it does more than save evaluations: in Part II it attains the *best* objective ($F = 0.569$) using only 80 evaluations, so it is the retained default; in Part I it reaches the optimal corner in the fewest evaluations as well. Bayesian optimization is therefore the recommended method for the expensive

parametric problem, where the number of PDE solves is the binding constraint.

9 Conclusion and Possible Improvements

We built an automatic design-optimization platform for HEAT-COND satisfying all required tasks: a geometry-aligned FreeFEM++ P_1 solver, an in-solver objective module, an optimization driver, a graphical interface and command-line pipeline, and a structured numerical analysis. We then went beyond the required scope with a fully enriched, multi-objective design problem.

Part I establishes that the benchmark objective is smooth and unimodal with a boundary optimum $\mathbf{x}^\star = (1, 1, 1, 1, 1, 0.01)$, $J^\star \approx 0.730$, on which Nelder–Mead is the most efficient optimizer. **Part II** enriches the design with material choice (price, density) and fin dimensioning, maximizing a performance/cost/mass objective. Applying the same methods plus Bayesian optimization, the four perform within a few percent on this 16-dimensional problem, and **Bayesian optimization attains the best objective using by far the fewest evaluations**, so it is retained as the default; its convergence is well behaved both per evaluation and in CPU time, and the performance–cost Pareto front shows the material cost can be cut by about 90% for a moderate performance loss. The other three methods stay within a few percent, the Adam→L-BFGS hybrid a close second.

Possible improvements include: (i) scaling the Bayesian-optimization study demonstrated here to mixed-variable, higher-dimensional surrogates, to further cut the number of PDE solves; (ii) an *adjoint gradient* for the smooth Part I sub-problem, which would make a gradient method the cheapest option; and (iii) a richer material catalogue and manufacturing constraints to bring the enriched problem closer to a real heat-sink design.

Member Contributions

Alexandre Le Droucpeet built the geometry and meshing (the FreeFEM++ `mesh.edp` and `mesh_param.edp`, and the comb-domain construction) and implemented the optimization methods. Laurent Zhu carried out the comparative studies of the methods and built the graphical interface. Both authors wrote the report.

References

[1] H. Ballout, Y. Maday, C. Prud'homme. *Nonlinear compressive reduced basis approximation for multi-parameter elliptic problem.* In *Multiscale, Nonlinear and Adaptive Approximation II*, pp. 55–73. Springer, 2024.